

Laboratorio de Automatizacion SOC

Guia Completa de Instalacion desde Cero

Suricata IDS · Elasticsearch · Kibana · Filebeat · n8n · Telegram

Stack: Suricata 8.x + ELK Stack 8.x + n8n + Python + Flask + ngrok

Plataforma: Kali Linux VM (VirtualBox)

Autor: Marcos — @Markussella

GitHub: github.com/Markussella/blue-team-lab

Version: 1.0 — 2026

Proyecto de portfolio de ciberseguridad — Blue Team / SOC Analyst

Indice

1. Descripcion del Proyecto
2. Arquitectura
3. Requisitos Previos
4. Fase 1 — Instalacion y Configuracion de Suricata IDS
5. Fase 2 — ELK Stack (Elasticsearch + Kibana + Filebeat)
6. Fase 3 — Firewall Mock (API Flask)
7. Fase 4 — Tunel ngrok
8. Fase 5 — Workflow SOAR en n8n
9. Fase 6 — Bot de Telegram
10. Fase 7 — Suricata Watcher (Python)
11. Prueba End-to-End
12. Solucion de Problemas
13. Comandos de Arranque

1. Descripcion del Proyecto

Laboratorio SOC funcional desplegado en una VM Kali Linux. El sistema monitoriza el trafico de red en tiempo real con Suricata IDS (+49.000 reglas), indexa eventos en Elasticsearch mediante Filebeat, y dispara flujos de respuesta automatica a incidentes a traves de n8n — incluyendo bloqueo de IPs, registro de incidentes, alertas por email y notificaciones por Telegram.

Pipeline de Respuesta Automatica:

- > Suricata detecta una amenaza en la red y escribe la alerta en /var/log/suricata/eve.json
- > Filebeat lee el archivo eve.json y lo indexa en Elasticsearch
- > El script Python Watcher consulta Elasticsearch cada 60 segundos buscando alertas nuevas
- > Cuando encuentra una alerta, la envia al webhook de n8n via POST
- > n8n normaliza los campos y evalua la severidad
- > Si severidad es alta/critica: bloquea la IP + registra el incidente + envia email + envia Telegram
- > Si severidad es baja/media: registra el evento en el indice soar-low-severity

2. Arquitectura

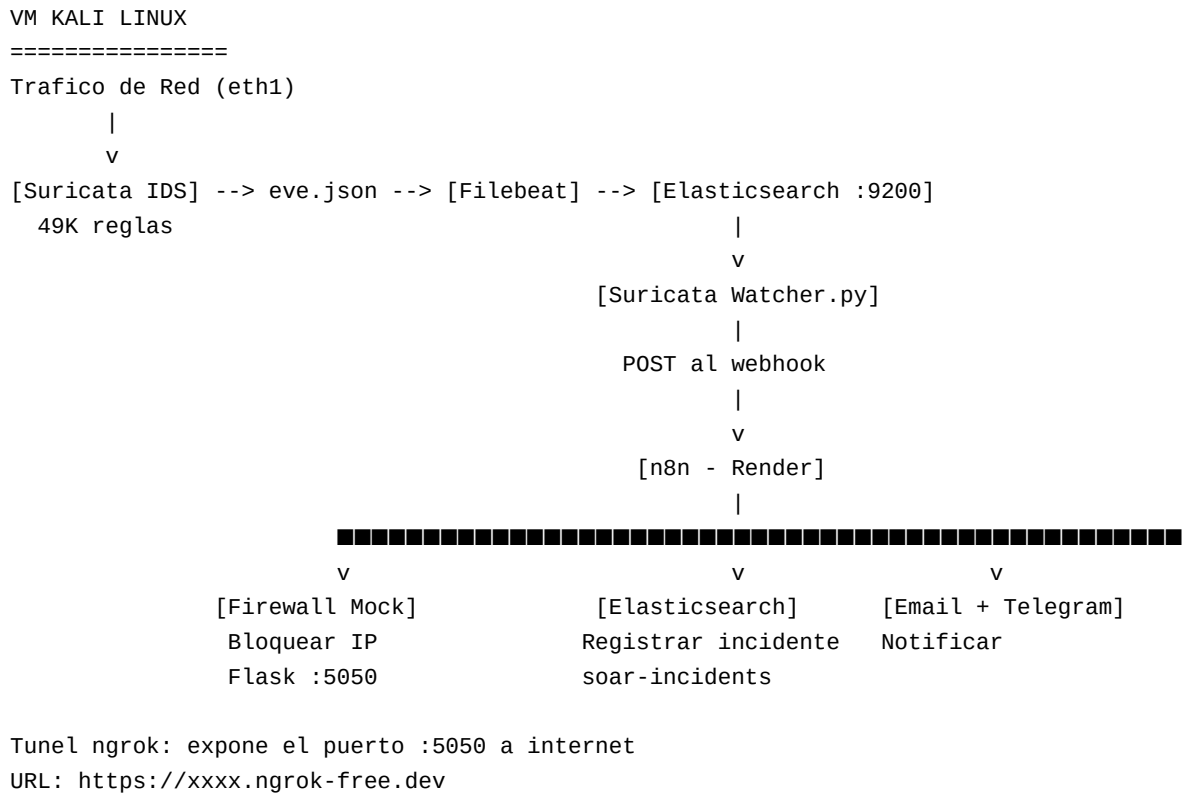


Tabla de Componentes:

Suricata IDS — Deteccion de amenazas en la red | Suricata 8.0.3 + ET Open Rules (49K)

Filebeat — Envío de logs a Elasticsearch | Filebeat 8.19

Elasticsearch — Almacenamiento y búsqueda de eventos | Elasticsearch 8.x

Kibana — Dashboard de visualizacion | *Kibana 8.x*

Suricata Watcher — Script de consulta de alertas | Python 3

n8n — Orquestacion SOAR | *n8n (Render cloud)*

Firewall Mock — Simulación de bloqueo de IPs | Flask (Python)

ngrok — Exponer puerto local a internet | *ngrok plan gratuito*

Mailtrap — Prueba de alertas por email | SMTP sandbox

Bot de Telegram — Notificaciones instantaneas al movil | *Telegram API*

3. Requisitos Previos

Hardware y Software:

- VirtualBox con Kali Linux VM (minimo 4GB de RAM recomendado)
- 2 interfaces de red: NAT (eth0) + Bridged o Host-Only (eth1)
- Conexion a internet desde la VM
- PC/Mac con Windows para las pruebas desde PowerShell

Cuentas necesarias (todas gratuitas):

- ngrok — <https://ngrok.com> (plan gratuito)
- n8n — <https://n8n.io> (desplegado en Render, plan gratuito)
- Mailtrap — <https://mailtrap.io> (sandbox gratuito)
- Telegram — app movil + crear bot con @BotFather

4. Fase 1 — Instalacion y Configuracion de Suricata IDS

4.1 Instalar Suricata

```
sudo apt update && sudo apt install suricata suricata-update -y
```

4.2 Actualizar las Reglas

Descargar +49.000 reglas de Emerging Threats Open:

```
sudo suricata-update
```

4.3 Identificar la Interfaz de Red Correcta

Hay que encontrar que interfaz lleva el trafico real de la red local. Normalmente es eth1 en VMs con modo Bridged:

```
ip addr show
# Busca tu IP de red local (ej: 192.168.1.x), normalmente en eth1
```

4.4 Corregir el Override de systemd

CRITICO: El override por defecto incluye el flag **-S analyst.rules** que hace que Suricata ignore completamente el archivo yaml y solo cargue las reglas personalizadas (11 reglas). Hay que eliminarlo:

```
sudo nano /etc/systemd/system/suricata.service.d/override.conf
```

Establecer el contenido a:

```
[Service]
ExecStart=
ExecStart=/usr/bin/suricata -c /etc/suricata/suricata.yaml -i eth1 --pidfile /var/run/suricata/suricata.pid
PIDFile=/var/run/suricata/suricata.pid
Restart=on-failure
```

4.5 Configurar suricata.yaml

Verificar que la seccion rule-files apunta al archivo correcto. Buscar alrededor de la linea 2308:

```
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
```

Importante: La indentacion debe ser exactamente 0 espacios para default-rule-path y rule-files, y 2 espacios para los elementos de la lista. Si hay errores de indentacion, Suricata no arrancara.

Si el yaml queda corrupto: Restaurar el backup original con:

```
sudo cp /etc/suricata/suricata.yaml.dpkg-dist /etc/suricata/suricata.yaml
```

4.6 Arrancar y Verificar

```
sudo systemctl daemon-reload
sudo systemctl restart suricata
sleep 15
sudo suricatasc -c "ruleset-stats"
```

```
# Salida esperada:  
# {"message":[{"id":0,"rules_loaded":49023,...}],"return":"OK"}
```

4.7 Añadir Reglas Personalizadas (Opcional)

Crear el archivo `/var/lib/suricata/rules/analyst.rules` con reglas propias:

```
alert icmp any any -> $HOME_NET any (msg:"LOCAL SCAN ICMP Echo Request"; itype:8; icode:0; classtype:icmp-echo;  
alert tcp any any -> $HOME_NET 22 (msg:"LOCAL ALERT SSH Brute Force"; flow:to_server,established;  
alert http any any -> $HOME_NET any (msg:"LOCAL ALERT SQL Injection"; http.uri; content:"union se"
```

Luego añadir en la sección `rule-files` del `yml`: - **analyst.rules**

5. Fase 2 — ELK Stack (Elasticsearch + Kibana + Filebeat)

5.1 Instalar Elasticsearch

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/debian/apt/pool elasticsearch -y" | sudo tee /etc/apt/sources.list.d/elasticsearch.list
sudo apt update && sudo apt install elasticsearch -y
```

Deshabilitar la seguridad para el laboratorio (simplifica la configuracion):

```
sudo nano /etc/elasticsearch/elasticsearch.yml
# Anadir al final:
xpack.security.enabled: false
xpack.security.http.ssl.enabled: false

sudo systemctl enable --now elasticsearch
# Verificar que funciona:
curl http://localhost:9200
```

5.2 Instalar Kibana

```
sudo apt install kibana -y
sudo nano /etc/kibana/kibana.yml
# Configurar:
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://localhost:9200"]

sudo systemctl enable --now kibana
# Acceder en: http://localhost:5601
```

5.3 Instalar y Configurar Filebeat

```
sudo apt install filebeat -y
```

Editar /etc/filebeat/filebeat.yml para leer el eve.json de Suricata:

```
filebeat.inputs:
- type: filestream
  id: suricata-eve
  enabled: true
  paths:
  - /var/log/suricata/eve.json
  parsers:
  - ndjson:
    target: ""
    add_error_key: true

output.elasticsearch:
  hosts: ["http://localhost:9200"]
  index: "filebeat-%[agent.version]}-%{yyyy.MM.dd}"

sudo systemctl enable --now filebeat
# Verificar que se creo el indice:
curl http://localhost:9200/_cat/indices?v | grep filebeat
```


6. Fase 3 — Firewall Mock API (Flask)

Este script simula un firewall real con una API REST. Tambien actua como proxy hacia Elasticsearch para evitar necesitar un segundo tunel ngrok.

Crear el archivo ~/soar-lab/firewall_api_mock.py:

```
from flask import Flask, request, jsonify
import requests, json
from datetime import datetime

app = Flask(__name__)
blocked_ips = {}
AUTH_TOKEN = "CAMBIAR_POR_TU_TOKEN_FIREWALL"

@app.route('/block-ip', methods=['POST'])
def block_ip():
    token = request.headers.get('X-Auth-Token')
    if token != AUTH_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401
    data = request.json
    ip = data.get('ip')
    reason = data.get('reason', 'Sin motivo')
    blocked_ips[ip] = {"reason": reason, "timestamp": str(datetime.now())}
    print(f"[FIREWALL] IP Bloqueada: {ip} | Motivo: {reason}")
    return jsonify({"status": "success", "ip": ip, "action": "blocked"})

@app.route('/blocked', methods=['GET'])
def list_blocked():
    return jsonify({"blocked_ips": blocked_ips, "count": len(blocked_ips)})

# Proxy a Elasticsearch
@app.route('/elastic/<path:path>', methods=['GET', 'POST', 'PUT'])
def elastic_proxy(path):
    url = f"http://localhost:9200/{path}"
    resp = requests.request(method=request.method, url=url,
        headers={"Content-Type": "application/json"},
        data=request.get_data())
    return resp.content, resp.status_code

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5050, debug=True)

pip3 install flask requests
python3 ~/soar-lab/firewall_api_mock.py
```

Nota: Mantener este script corriendo en una terminal dedicada. No usar Ctrl+Z para mandarlo al fondo — usar una terminal separada.

7. Fase 4 — Tunnel ngrok

ngrok expone el puerto 5050 de Kali (donde corre el Firewall Mock) a internet para que n8n pueda alcanzarlo desde la nube.

7.1 Instalar ngrok

```
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc &&
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list &&
sudo apt update && sudo apt install ngrok -y
```

7.2 Autenticarse con el token

```
ngrok config add-authtoken TU_TOKEN_DEL_DASHBOARD_NGROK
```

7.3 Iniciar el tunnel

```
ngrok http 5050
# Anotar la URL: https://xxxx.ngrok-free.dev
# Esta URL hay que actualizarla en n8n cada vez que cambie
```

Importante: El plan gratuito de ngrok genera una URL nueva cada vez que reinicias el agente. Cuando esto ocurra, hay que actualizar todos los nodos HTTP de n8n con la nueva URL.

Consejo: Siempre añadir la cabecera **ngrok-skip-browser-warning: 1** en todas las peticiones HTTP que pasen por ngrok, de lo contrario devolverá una página HTML en lugar del JSON esperado.

8. Fase 5 — Workflow SOAR en n8n

8.1 Crear cuenta en n8n via Render

- Ir a <https://render.com> y crear cuenta gratuita
- Desplegar n8n como Web Service usando la imagen oficial de Docker de n8n
- Anotar la URL de n8n: <https://tu-app.onrender.com>
- Entrar en n8n y crear un nuevo workflow llamado 'SOAR Suricata'

8.2 Construir los Nodos del Workflow en este orden:

Nodo 1 — Webhook (Trigger)

Metodo: POST
Ruta: suricata-alert
Respuesta: Last Node
Cuerpo de respuesta: Alerta recibida correctamente

Nodo 2 — Normalizar Alerta (Code (JavaScript))

```
const body = $input.first().json.body || $input.first().json;  
return [{  
  json: {  
    rule_name: body.rule_name || body['rule.name'] || 'Unknown',  
    severity: body.severity || 'medium',  
    source_ip: body.source_ip || body.src_ip || '0.0.0.0',  
    destination_ip: body.destination_ip || body.dest_ip || '0.0.0.0',  
    timestamp: body.timestamp || new Date().toISOString(),  
    event_module: body.event_module || 'suricata',  
    event_action: body.event_action || 'alert'  
  }  
}];
```

Nodo 3 — If (Alta Severidad) (IF)

Condicion 1: {{ \$json.severity }} equals high
Condicion 2: {{ \$json.severity }} equals critical
Combinar: OR | Convertir tipos: ON

Nodo 4 — Bloquear IP (Firewall) (HTTP Request [rama TRUE])

Metodo: POST
URL: <https://TU-URL-NGROK/block-ip>
Cabecera: X-Auth-Token: CAMBIAR_POR_TU_TOKEN_FIREWALL
Cabecera: ngrok-skip-browser-warning: 1
Cuerpo (JSON): {"ip": "{{ \$('Normalizar Alerta').item.json.source_ip }}", "reason": "{{ \$('Normalizar Alerta').item.json.severity }}"}

Nodo 5 — Registrar Incidente (Alta) (HTTP Request [rama TRUE])

Metodo: POST
URL: https://TU-URL-NGROK/elastic/soar-incidents/_doc
Cabecera: ngrok-skip-browser-warning: 1
Cuerpo: todos los campos del nodo Normalizar Alerta

Nodo 6 — Notificar Email (Send Email [rama TRUE])

Credenciales SMTP: Mailtrap (sandbox.smtp.mailtrap.io:2525, SSL off)
De/Para: tu@email.com
Asunto: [SOAR ALERT] {{ \$('Normalizar Alerta').item.json.rule_name }}
Cuerpo: HTML con todos los campos de la alerta

Nodo 7 — Notificar Telegram (Telegram [rama TRUE])

Credencial: token de tu bot
Chat ID: tu chat ID
Mensaje: plantilla con emoji y campos de la alerta

Nodo 8 — Registrar Evento (Baja) (HTTP Request [rama FALSE])

Metodo: POST
URL: https://TU-URL-NGROK/elastic/soar-low-severity/_doc
Cuerpo: todos los campos del nodo Normalizar Alerta

8.3 Activar el Workflow

Hacer clic en el toggle de la esquina superior derecha para poner el workflow en estado Activo. Sin este paso el webhook no recibirá peticiones.

9. Fase 6 — Bot de Telegram

9.1 Crear el Bot

- Abrir Telegram y buscar @BotFather
- Enviar /newbot
- Elegir nombre: SOAR Alert Bot
- Elegir username (debe terminar en 'bot'): soar_alert_XXXX_bot
- Guardar el token que proporciona BotFather

9.2 Obtener el Chat ID

```
# 1. Buscar el bot en Telegram y enviarle /start
# 2. Abrir en el navegador:
https://api.telegram.org/botTU_TOKEN/getUpdates
# 3. Buscar: "chat":{"id":XXXXXXXXX}
# Ese numero es tu Chat ID
```

9.3 Probar el Bot

```
curl "https://api.telegram.org/botTU_TOKEN/sendMessage?chat_id=TU_CHAT_ID&text=Prueba+SOAR"
```

9.4 Plantilla del Mensaje en n8n

```
Alerta SOAR
Regla: {{ $('Normalizar Alerta').item.json.rule_name }}
Severidad: {{ $('Normalizar Alerta').item.json.severity }}
IP Origen: {{ $('Normalizar Alerta').item.json.source_ip }}
IP Destino: {{ $('Normalizar Alerta').item.json.destination_ip }}
Timestamp: {{ $('Normalizar Alerta').item.json.timestamp }}
```

10. Fase 7 — Suricata Watcher (Python)

Este script es el puente entre Elasticsearch y n8n. Consulta Elasticsearch cada 60 segundos buscando alertas nuevas de Suricata y las envia al webhook de n8n automaticamente.

Crear ~/soar-lab/suricata_watcher.py:

```
import requests
import json
import time
from datetime import datetime, timedelta, timezone

ELASTICSEARCH_URL = "http://localhost:9200"
INDEX = ".ds-filebeat-*"
N8N_WEBHOOK = "https://TU-URL-N8N/webhook/suricata-alert"
POLL_INTERVAL = 60 # segundos

def buscar_alertas():
    ahora = datetime.now(timezone.utc)
    desde = (ahora - timedelta(minutes=2)).strftime('%Y-%m-%dT%H:%M:%SZ')

    query = {
        "query": {
            "bool": {
                "must": [
                    {"match": {"event_type": "alert"}},
                    {"range": {"@timestamp": {"gte": desde}}}
                ]
            }
        },
        "sort": [{"@timestamp": "desc"}],
        "size": 10
    }

    response = requests.post(
        f"{ELASTICSEARCH_URL}/{INDEX}/_search",
        headers={"Content-Type": "application/json"},
        data=json.dumps(query)
    )
    return response.json().get("hits", {}).get("hits", [])

def enviar_a_n8n(alerta):
    source = alerta.get("_source", {})
    alert_data = source.get("alert", {})

    severidades = ["critical", "high", "medium", "low"]
    sev_idx = min(alert_data.get("severity", 3) - 1, 3)

    payload = {
        "rule_name": alert_data.get("signature", "Unknown"),
        "severity": severidades[sev_idx],
        "source_ip": source.get("src_ip", "0.0.0.0"),
        "destination_ip": source.get("dest_ip", "0.0.0.0"),
        "event_module": "suricata",
    }
```

```

        "event_action": "alert",
        "timestamp": source.get("timestamp", datetime.now().isoformat())
    }

    response = requests.post(N8N_WEBHOOK,
        headers={"Content-Type": "application/json"},
        data=json.dumps(payload))
    return response.status_code

if __name__ == "__main__":
    print("[Watcher] Iniciando Suricata Alert Watcher...")
    while True:
        alertas = buscar_alertas()
        if alertas:
            for alerta in alertas:
                status = enviar_a_n8n(alerta)
                sig = alerta["_source"].get("alert", {}).get("signature", "Unknown")
                print(f"[Alerta enviada -> n8n ({status})] {sig}")
            else:
                print(f"[{datetime.now().strftime('%H:%M:%S')}] Sin alertas nuevas")
                time.sleep(POLL_INTERVAL)

python3 ~/soar-lab/suricata_watcher.py

```

11. Prueba End-to-End

11.1 Prueba Manual desde PowerShell (Windows)

```
Invoke-RestMethod -Uri "https://TU-URL-N8N/webhook/suricata-alert" `
-Method POST -ContentType "application/json" -Body '{
  "rule_name": "ET SCAN Nmap",
  "severity": "high",
  "source_ip": "192.168.1.100",
  "destination_ip": "10.0.0.1",
  "event_module": "suricata",
  "event_action": "alert",
  "timestamp": "2026-03-12T12:00:00Z"
}'
```

11.2 Prueba con Trafico Real de Red

```
# Terminal 1 - Monitorizar alertas en tiempo real
sudo tail -f /var/log/suricata/eve.json | grep 'event_type.*alert'

# Terminal 2 - Generar trafico malicioso
curl http://testmyids.org/uid/index.html
# o tambien:
sudo nmap -sS 192.168.1.134 # (tu propia IP)
```

11.3 Verificar Resultados

```
# IPs bloqueadas por el firewall
curl http://localhost:5050/blocked | python3 -m json.tool

# Incidentes en Elasticsearch
curl 'http://localhost:9200/soar-incidents/_search?pretty' | grep source_ip

# Reglas cargadas en Suricata
sudo suricatasc -c "ruleset-stats"
# Esperado: rules_loaded: 49023
```

11.4 Resultados Esperados del Flujo Completo

- ✓ Webhook de n8n responde 'Alerta recibida correctamente'
- ✓ La IP aparece en el endpoint /blocked del firewall
- ✓ El incidente queda registrado en el indice soar-incidents de Elasticsearch
- ✓ Se recibe email en la bandeja de Mailtrap
- ✓ Llego mensaje de alerta al movil por Telegram
- ✓ En n8n > Executions aparece la ejecucion con todos los nodos en verde

12. Solucion de Problemas

Problema: Suricata solo carga 11 reglas

Verificar el override de systemd:
sudo systemctl cat suricata | grep ExecStart
Eliminar el flag -S analyst.rules del override.conf
Verificar que suricata.yaml tiene rule-files apuntando a suricata.rules

Problema: Suricata no arranca tras editar el yaml

Ejecutar el test de configuracion:
sudo suricata -T -c /etc/suricata/suricata.yaml 2>&1 | grep -i error
Revisar la indentacion en rule-files (0 espacios para claves, 2 para lista)
Restaurar backup: sudo cp /etc/suricata/suricata.yaml.dpkg-dist /etc/suricata/suricata.yaml

Problema: La URL de ngrok cambia y n8n no puede contactar el firewall

Actualizar TODOS los nodos HTTP Request de n8n con la nueva URL de ngrok
Siempre anadir la cabecera: ngrok-skip-browser-warning: 1
Considerar el plan de pago de ngrok para tener URL estatica

Problema: El puerto 5050 ya esta en uso

sudo fuser -k 5050/tcp

Problema: El firewall mock deja de responder

Comprobar si sigue corriendo: ps aux | grep firewall
Reiniciarlo: python3 ~/soar-lab/firewall_api_mock.py

Problema: El watcher siempre muestra 'Sin alertas nuevas'

Verificar que el campo event_type existe en los documentos:
curl localhost:9200/.ds-filebeat-*/_search -d '{"query":{"match":{"event_type":"alert"}}}'
Comprobar diferencia de zona horaria entre el sistema y Elasticsearch
Generar trafico nuevo: curl http://testmynids.org/uid/index.html

Problema: n8n tarda mucho en responder (Render)

El plan gratuito de Render deja el servicio en suspension tras 15 min de inactividad
La primera peticion puede tardar 30-60 segundos en despertar
Enviar una peticion de prueba y esperar antes de comprobar resultados

Problema: Elasticsearch no responde

sudo systemctl status elasticsearch
curl http://localhost:9200
Verificar que xpack.security.enabled: false esta en elasticsearch.yml

13. Comandos de Arranque

Ejecutar en este orden exacto cada vez que se inicia el laboratorio:

Paso 1 — Verificar que los servicios principales estan activos:

```
sudo systemctl status elasticsearch kibana filebeat suricata
# Todos deben mostrar: active (running)
```

Paso 2 — Iniciar el Firewall Mock (terminal dedicada):

```
cd ~/soar-lab && python3 firewall_api_mock.py
```

Paso 3 — Iniciar el tunel ngrok (nueva terminal):

```
ngrok http 5050
# Copiar la URL: https://xxxx.ngrok-free.dev
```

Paso 4 — Actualizar n8n con la nueva URL de ngrok (si ha cambiado):

Ir a n8n > editar cada nodo HTTP Request > actualizar la URL > guardar > publicar el workflow

Paso 5 — Iniciar el Suricata Watcher (nueva terminal):

```
python3 ~/soar-lab/suricata_watcher.py
```

Verificacion Rapida del Estado del Lab

```
# Estado de servicios
sudo systemctl is-active elasticsearch kibana filebeat suricata

# Reglas cargadas en Suricata
sudo suricatasc -c "ruleset-stats"

# Alertas recientes en Elasticsearch
curl -s 'localhost:9200/.ds-filebeat-*/_count' \
  -H 'Content-Type: application/json' \
  -d '{"query":{"match":{"event_type":"alert"}}}'

# IPs bloqueadas
curl http://localhost:5050/blocked
```

Lab Completado

Pipeline completo operativo:

- ✓ Suricata IDS — 49.023 reglas activas
- ✓ Filebeat -> Elasticsearch — eventos indexados
- ✓ Kibana — visualizacion lista
- ✓ Firewall Mock — bloqueo de IPs operativo
- ✓ ngrok — tunel activo
- ✓ n8n — workflow SOAR publicado
- ✓ Email (Mailtrap) — notificaciones funcionando
- ✓ Bot de Telegram — alertas instantaneas al movil
- ✓ Suricata Watcher — pipeline automatizado activo
- ✓ Tiempo de respuesta end-to-end: menos de 60 segundos